



Mata Kuliah : Pemrograman Web Lanjut (PWL)
Program Studi : D4 – Teknik Informatika
Semester : 4

Kelas : 2A
NIM : 244107020030
Nama : Athaulla Hafizh
Jobsheet Ke- : 2

Laporan Jobsheet 02

Link Repository : <https://github.com/athfizh/Pemrograman-Web-Lanjut>

Praktikum 01

Langkah	Jawaban/Deskripsi												
1	Pada bagian ini, kita akan membuat dua buah route dengan ketentuan sebagai berikut. <table><tr><th>No</th><th>Http Verb</th><th>Url</th><th>Fungsi</th></tr><tr><td>1</td><td>get</td><td>/hello</td><td>Tampilkan String Hello ke browser.</td></tr><tr><td>2</td><td>get</td><td>/world</td><td>Tampilkan String World ke browser</td></tr></table> Kita akan menggunakan project minggu sebelumnya yaitu PWL_2026.	No	Http Verb	Url	Fungsi	1	get	/hello	Tampilkan String Hello ke browser.	2	get	/world	Tampilkan String World ke browser
No	Http Verb	Url	Fungsi										
1	get	/hello	Tampilkan String Hello ke browser.										
2	get	/world	Tampilkan String World ke browser										
2	Buka file routes/web.php. Tambahkan sebuah route untuk nomor 1 seperti di bawah ini: <pre>Route::get('/hello', function () { return 'Hello World'; });</pre>												
3	Buka browser, tuliskan URL untuk memanggil route tersebut: localhost/PWL_2024/public/hello. Perhatikan halaman yang muncul apakah sudah sesuai dan jelaskan pengamatan Anda. Hello World												
4	Untuk membuat route kedua, tambahkan route /world seperti di bawah ini:												



	<pre>Route::get('/world', function () { return 'World'; });</pre>
5	Bukalah pada browser, tuliskan URL untuk memanggil route tersebut: localhost/PWL_2024/public/world. Perhatikan halaman yang muncul apakah sudah sesuai dan jelaskan pengamatan Anda. World Pengamatan (/hello & /world): Saat diakses melalui browser (contoh: localhost:8000/hello), browser akan menampilkan teks "Hello World" dan "World". Hal ini sesuai dengan instruksi karena fungsi <i>callback</i> secara langsung me-return string tersebut ke browser.
6	Selanjutnya, cobalah membuat route '/' yang menampilkan pesan 'Selamat Datang'. <pre>Route::get('/', function () { return 'Selamat Datang'; });</pre>
7	Kemudian buatlah route '/about' yang akan menampilkan NIM dan nama Anda. <pre>Route::get('/about', function () { return 'NIM: 244107020030, Nama: Athaulla Hafizh'; });</pre>
8	Kita akan memanggil route <code>/user/{name}</code> sekaligus mengirimkan parameter berupa nama user \$name seperti kode di bawah ini. <pre>Route::get('/user/{name}', function (\$name) { return 'Hafizh'. \$name; });</pre>
9	Jalankan kode dengan menuliskan URL untuk memanggil route tersebut: localhost/PWL_2024/public/user>NamaAnda. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Nama saya Athaulla



	Pengamatan (/user/Athaulla): Halaman akan menampilkan teks "Nama saya Athaulla". Route berhasil menangkap nilai dari URL dan memasukkannya ke dalam variabel \$name.
10	Selanjutnya, coba tuliskan URL: localhost/PWL_2024/public/user/. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda  Pengamatan (/user/ tanpa nama): Akan muncul pesan <i>error 404 Not Found</i>. Ini terjadi karena rute mewajibkan adanya data di URL, namun kita tidak mengirimkan apa pun.
11	Suatu route, juga bisa menerima lebih dari 1 parameter seperti kode berikut ini. Route menerima parameter \$postId dan juga \$comment. <pre>Route::get('/posts/{post}/comments/{comment}', function (\$postId, \$commentId) { return 'Pos ke-' . \$postId . " Komentar ke-: " . \$commentId; });</pre>
12	Jalankan kode dengan menuliskan URL untuk memanggil route tersebut: localhost/PWL_2024/public/posts/1/comments/5. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Pos ke-1 Komentar ke-: 5 Pengamatan (/posts/1/comments/5): Menampilkan "Pos ke-1 Komentar ke-: 5". Route terbukti mampu menangkap dan memproses lebih dari satu parameter secara bersamaan dari URL.
13	Kemudian buatlah route <code>/articles/{id}</code> yang akan menampilkan output “Halaman Artikel dengan ID {id}”, ganti id sesuai dengan input dari url. <pre>Route::get('/articles/{id}', function (\$id) { return "Halaman Artikel dengan ID " . \$id; });</pre>
14	Kita akan memanggil route <code>/user</code> sekaligus mengirimkan parameter berupa nama user \$name dimana parameternya bersifat opsional.

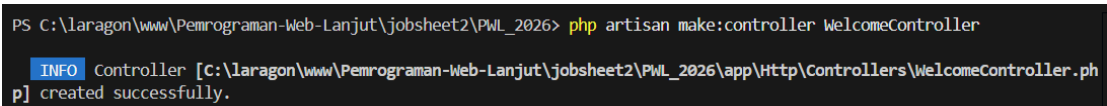
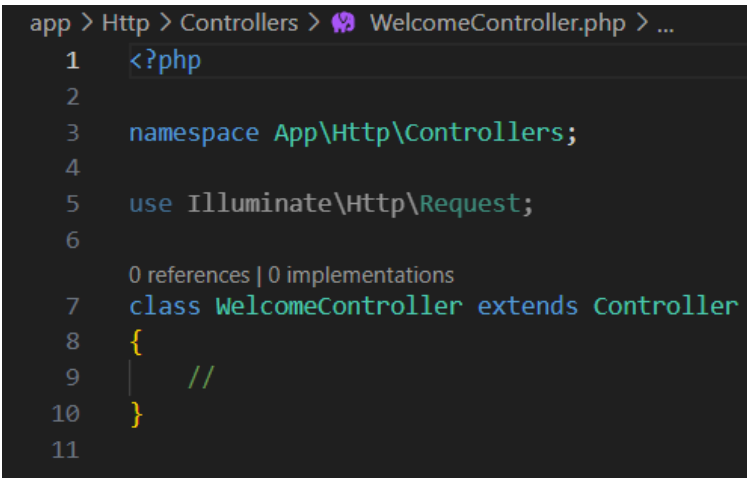
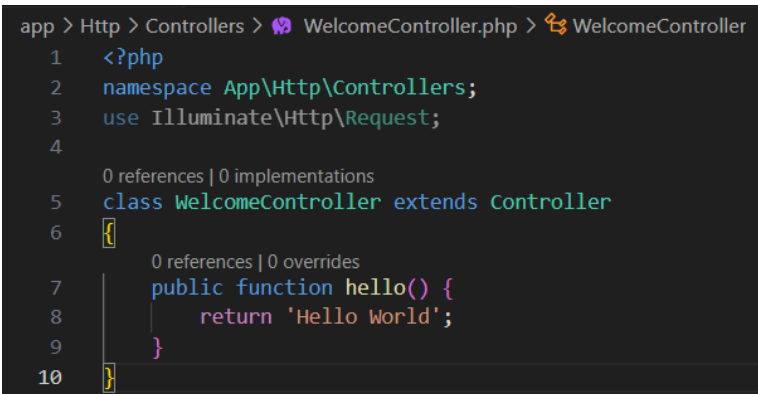


	<pre>Route::get('/user/{name?}', function (\$name = 'null') { return 'Nama saya ' . \$name; });</pre>
15	Jalankan kode dengan menuliskan URL: localhost/PWL_2024/public/user/. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Nama saya null Pengamatan (/user/ dengan optional parameter): Halaman tidak lagi error. Karena parameter bersifat opsional, jika kita tidak mengisi nama di URL, sistem akan menggunakan nilai <i>default</i> yang telah kita set (yaitu null), sehingga menampilkan "Nama saya null". Jika diisi /user/Athaulla, maka akan tetap menampilkan nama tersebut.
16	Selanjutnya tuliskan URL: localhost/PWL_2024/public/user>NamaAnda. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda Nama saya Athaulla Pengamatan (/user/Athaulla): Halaman akan menampilkan teks "Nama saya Athaulla". Route berhasil menangkap nilai dari URL dan memasukkannya ke dalam variabel \$name.
17	Ubah kode pada route /user menjadi seperti di bawah ini. <pre>Route::get('/user/{name?}', function (\$name = 'John') { return 'Nama saya ' . \$name; });</pre>
18	Jalankan kode dengan menuliskan URL: localhost/PWL_2024/public/user/. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Nama saya John Halaman tersebut tidak <i>error</i> karena kita telah menambahkan tanda tanya (?) pada {name?}, yang memberi tahu Laravel bahwa parameter nama tersebut bersifat opsional atau tidak wajib diisi. Karena tidak mengirimkan nama dari URL, maka program akan menggunakan nilai <i>default</i> yang sudah ditentukan di dalam fungsi, yaitu \$name='John'.



	+1 Sebaliknya, jika Anda mengakses /user/Athaulla, maka teks "John" akan tertimpa dan berubah menjadi "Nama saya Athaulla".
--	--

Praktikum 02

Langkah	Jawaban/Deskripsi
1	Untuk membuat controller pada Laravel telah disediakan perintah untuk menggenerate struktur dasarnya. Kita dapat menggunakan perintah artisan diikuti dengan definisi nama controller yang akan dibuat. 
2	Buka file pada app/Http/Controllers/WelcomeController.php. Struktur pada controller dapat digambarkan sebagai berikut: 
3	Untuk mendefinisikan action, silahkan tambahkan function dengan access public. Sehingga controller di atas menjadi sebagai berikut: 



4	Setelah sebuah controller telah didefinisikan action, kita dapat menambahkan controller tersebut pada route. Ubah route /hello menjadi seperti berikut: <pre>use App\Http\Controllers\WelcomeController; Route::get('/hello', [WelcomeController::class, 'hello']);</pre>
5	Buka browser, tuliskan URL untuk memanggil route tersebut: localhost/PWL_2024/public/hello. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Hello World Pengamatan: Hasil yang ditampilkan di browser sama persis yaitu "Hello World". Bedanya, arsitektur kode sekarang lebih terstruktur karena pemrosesan dipindahkan ke Controller.
6	Modifikasi dengan Konsep PageController Pertama, kita kumpulkan logika rute dasar ke dalam satu kelas bernama PageController. <pre>PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026> php artisan make:controller PageController INFO Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026\app\Http\Controllers\PageController.php] created successfully.</pre> Buka file app/Http/Controllers/PageController.php dan isikan fungsi-fungsi berikut: <pre>app > Http > Controllers > PageController.php > PageController 1 <?php 2 3 namespace App\Http\Controllers; 4 5 use Illuminate\Http\Request; 6 7 0 references 0 implementations 8 class PageController extends Controller 9 { 10 0 references 0 overrides 11 public function index() { 12 return 'Selamat Datang'; 13 } 14 15 0 references 0 overrides 16 public function about() { 17 return 'NIM: 2241720000, Nama: Athaula Hafizh'; 18 } 19 20 0 references 0 overrides 21 public function articles(\$id) { 22 return 'Halaman Artikel dengan Id ' . \$id; 23 } 24 }</pre> Lalu, modifikasi file routes/web.php untuk memanggil fungsi-fungsi tersebut: <pre>use App\Http\Controllers\PageController;</pre>



	<pre>Route::get('/', [PageController::class, 'index']); Route::get('/about', [PageController::class, 'about']); Route::get('/articles/{id}', [PageController::class, 'articles']);</pre>	
7	Modifikasi kembali implementasi sebelumnya dengan konsep Single Action Controller. Sehingga untuk hasil akhir yang didapatkan akan ada HomeController, AboutController dan ArticleController. Modifikasi juga route yang digunakan. <pre>PS C:\laragon\php artisan make:controller HomeController_2026> [INFO] Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026\app\Http\Controllers\HomeController.php] created successfully. PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026> php artisan make:controller AboutController [INFO] Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026\app\Http\Controllers\AboutController.php] created successfully. PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026> php artisan make:controller ArticleController [INFO] Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026\app\Http\Controllers\ArticleController.php] created successfully.</pre> Isi masing-masing file <i>Controller</i> di folder app/Http/Controllers: - HomeController.php: <pre>app > Http > Controllers > HomeController.php > HomeController.php 1 <?php 2 3 namespace App\Http\Controllers; 4 5 use Illuminate\Http\Request; 6 7 class HomeController extends Controller { 8 public function index() { 9 return 'Selamat Datang'; 10 } 11 }</pre> - AboutController.php:	



	<pre>app > Http > Controllers > AboutController.php > ... 1 <?php 2 3 namespace App\Http\Controllers; 4 5 use Illuminate\Http\Request; 6 7 0 references 0 implementations 8 class AboutController extends Controller 9 { 10 0 references 0 overrides 11 public function about() { 12 return 'NIM: 244107020030, Nama: Athaulla Hafizh'; 13 } 14 }</pre> - ArticleController.php: <pre>app > Http > Controllers > ArticleController.php > ... 1 <?php 2 3 namespace App\Http\Controllers; 4 5 use Illuminate\Http\Request; 6 7 0 references 0 implementations 8 class ArticleController extends Controller 9 { 10 0 references 0 overrides 11 public function articles(\$id) { 12 return 'Halaman Artikel dengan Id ' . \$id; 13 } 14 }</pre> Ubah kembali rute di routes/web.php untuk memanggil <i>Single Action Controller</i> ini: <pre>use App\Http\Controllers\HomeController; use App\Http\Controllers\AboutController; use App\Http\Controllers\ArticleController; Route::get('/', [HomeController::class, 'index']); Route::get('/about', [AboutController::class, 'about']); Route::get('/articles/{id}', [ArticleController::class, 'articles']);</pre>
8	Membuat <i>Resource Controller</i> Jalankan perintah berikut di terminal untuk <i>generate</i> sebuah <i>controller</i> dengan nama PhotoController yang otomatis berisi <i>method-method</i> standar untuk proses CRUD:



	<pre>PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026> php artisan make:controller PhotoController --resource</pre> <pre>INFO Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026\app\Http\Controllers\PhotoController.php] created successfully.</pre>
9	Mendaftarkan Route <i>Resource</i> Setelah controller berhasil <i>di-generate</i>, tambahkan kode program berikut pada file routes/web.php agar dapat terhubung dengan <i>frontend</i>: <pre>use App\Http\Controllers\PhotoController;</pre> <pre>Route::resource('photos', PhotoController::class);</pre>
10	Mengecek Daftar Route Jalankan perintah ini di terminal untuk melihat semua rute yang otomatis dibuat oleh Laravel untuk CRUD foto: <pre>PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\PWL_2026> php artisan route:list</pre> <pre>GET HEAD / GET HEAD about GET HEAD articles/{id} GET HEAD hello WelcomeController@hello GET HEAD posts/{post}/comments/{comment} GET HEAD storage/{path} storage.local PUT storage/{path} storage.local.upload GET HEAD up GET HEAD user/{name?} GET HEAD world</pre> Showing [10] routes Penjelasan: Ini membuktikan satu baris Route::resource berhasil membuat 7 rute berbeda sekaligus.
11	Pada route list semua route yang berhubungan untuk crud photo sudah di generate oleh laravel. Jika tidak semua route pada resource controller dibutuhkan dapat dikurangi dengan mengupdate route pada web.php menjadi seperti berikut ini. <pre>Route::resource('photos', PhotoController::class)->only(['index', 'show']);</pre> <pre>Route::resource('photos', PhotoController::class)->except(['create', 'store', 'update', 'destroy']);</pre>

Praktikum 03

Langkah	Jawaban/Deskripsi
1	Pada direktori app/resources/views, buatlah file hello.blade.php.



	<pre>resources > views >  hello.blade.php 1 <html> 2 <body> 3 <h1>Hello, {{ \$name }}</h1> 4 </body> 5 </html></pre>	
2	View tersebut dapat dijalankan melalui Routing, dimana <i>route</i> akan memanggil View sesuai dengan nama <i>file</i> tanpa 'blade.php'. (Catatan: Gantilah Andi dengan nama Anda) <pre>Route::get('/greeting', function () { return view('hello', ['name' => 'Athaulla']); });</pre>	
3	Jalankan code dengan membuka url localhost/PWL_2024/public/greeting. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Hello, Athaulla Pengamatan: Saat mengakses http://localhost:8000/greeting, browser menampilkan teks berukuran besar (H1) bertuliskan "Hello, Athaulla". Route secara langsung memanggil file View hello.blade.php dan berhasil mengirimkan data array ['name' => 'Athaulla'] yang kemudian dicetak oleh <i>Blade engine</i> menggunakan sintaks {{ \$name }}.	
4	Buatlah direktori blog di dalam direktori views. <pre>views └── blog</pre>	
5	Pindahkan file hello.blade.php ke dalam direktori blog. <pre>blog └──  hello.blade.php</pre>	
6	Selanjutnya lakukan perubahan pada route. <pre>Route::get('/greeting', function () { return view('blog.hello', ['name' => 'Athaulla']); });</pre> You, now • Uncommitted changes	
7	Jalankan code dengan membuka url localhost/PWL_2024/public/greeting. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda.	



	Hello, Athaulla Pengamatan: Hasil di browser tetap sama persis, yaitu menampilkan "Hello, Athaulla". Ini membuktikan bahwa penggunaan titik (blog.hello) berfungsi dengan baik untuk mereferensikan file hello.blade.php yang berada di dalam sub-direktori blog.
8	Buka WelcomeController.php dan tambahkan fungsi baru yaitu greeting. <pre><?php namespace App\Http\Controllers; use Illuminate\Http\Request; 2 references 0 implementations class WelcomeController extends Controller{ 1 reference 0 overrides public function hello(){ return 'Hello World'; } 0 references 0 overrides public function greeting(){ return view('blog.hello', ['name' => 'Athaulla']); } }</pre>
9	Ubah route /greeting dan arahkan ke WelcomeController pada fungsi greeting. <pre>Route::get('/greeting', [WelcomeController::class, 'greeting']);</pre>
10	Jalankan code dengan membuka url localhost/PWL_2024/public/greeting. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Hello, Athaulla Pengamatan: Tampilan di browser tidak berubah ("Hello, Athaulla"). Namun secara arsitektur, aplikasi sekarang sudah menerapkan konsep MVC yang benar. Route hanya bertugas menerima URL, lalu mendelegasikan tugasnya ke WelcomeController, dan Controller inilah yang memanggil View serta mengirimkan datanya.
11	Buka WelcomeController.php dan tambahkan ubah fungsi greeting.



	<pre>public function greeting() { return view('blog.hello') ->with('name', 'Athaulla') ->with('occupation', 'Astronaut'); }</pre>	
12	Ubah hello.blade.php agar dapat menampilkan dua parameter. <pre>resources > views > blog > hello.blade.php 1 <html> 2 3 <body> 4 <h1>Hello, {{ \$name }}</h1> 5 <h1>You are {{ \$occupation }}</h1> 6 </body> 7 8 </html></pre>	
13	Jalankan code dengan membuka url localhost/PWL_2024/public/greeting. Perhatikan halaman yang muncul dan jelaskan pengamatan Anda. Hello, Athaulla You are Astronaut Pengamatan: Saat halaman di- <i>refresh</i> , browser kini menampilkan dua baris teks: "Hello, Athaulla" dan "You are Astronaut". Fungsi ->with() terbukti sangat berguna untuk menambahkan/mengirimkan variabel data satu per satu ke dalam View dengan cara yang lebih mudah dibaca dibandingkan menggunakan satu array besar.	

Soal Praktikum

Langkah	Jawaban/Deskripsi
1	Jalankan Langkah-langkah Praktikum pada jobsheet di atas. Lakukan sinkronisasi perubahan pada project PWL_2024 ke Github.
2	Buatlah project baru dengan nama POS. Project ini merupakan sebuah aplikasi Point of Sales yang digunakan untuk membantu penjualan.



	▼ jobsheet2 > POS									
3	Buatlah beberapa route, controller, dan view sesuai dengan ketentuan sebagai berikut. <table><tr><td>1</td><td>Halaman Home Menampilkan halaman awal website</td></tr><tr><td>2</td><td>Halaman Products Menampilkan daftar product (route prefix) /category/food-beverage /category/beauty-health /category/home-care /category/baby-kid</td></tr><tr><td>3</td><td>Halaman User Menampilkan profil pengguna (route param) /user/{id}/name/{name}</td></tr><tr><td>4</td><td>Halaman Penjualan Menampilkan halaman transaksi POS</td></tr></table> Buat semua <i>Controller</i> yang dibutuhkan: <pre>PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS> php artisan make:controller HomeController INFO Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS\app\Http\Controllers\HomeController.php] created successfully. PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS> php artisan make:controller ProductController INFO Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS\app\Http\Controllers\ProductController.php] p] created successfully. PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS> php artisan make:controller UserController INFO Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS\app\Http\Controllers\UserController.php] created successfully. PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS> php artisan make:controller SalesController INFO Controller [C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet2\POS\app\Http\Controllers\SalesController.php] created successfully.</pre>	1	Halaman Home Menampilkan halaman awal website	2	Halaman Products Menampilkan daftar product (route prefix) /category/food-beverage /category/beauty-health /category/home-care /category/baby-kid	3	Halaman User Menampilkan profil pengguna (route param) /user/{id}/name/{name}	4	Halaman Penjualan Menampilkan halaman transaksi POS	
1	Halaman Home Menampilkan halaman awal website									
2	Halaman Products Menampilkan daftar product (route prefix) /category/food-beverage /category/beauty-health /category/home-care /category/baby-kid									
3	Halaman User Menampilkan profil pengguna (route param) /user/{id}/name/{name}									
4	Halaman Penjualan Menampilkan halaman transaksi POS									
4	Route tersebut menjalankan fungsi pada Controller yang berbeda di setiap halaman. Buka file POS/routes/web.php dan isikan arsitektur rute berikut sesuai tabel yang diminta:									



	<pre>web.php U x POS > routes > web.php 1 <?php 2 3 use Illuminate\Support\Facades\Route; 4 use App\Http\Controllers\HomeController; 5 use App\Http\Controllers\ProductController; 6 use App\Http\Controllers\UserController; 7 use App\Http\Controllers\SalesController; 8 9 // 1. Halaman Home 10 Route::get('/', [HomeController::class, 'index']); 11 12 // 2. Halaman Products (Route Prefix) 13 Route::prefix('category')->group(function () { 14 Route::get('/food-beverage', [ProductController::class, 'foodBeverage']); 15 Route::get('/beauty-health', [ProductController::class, 'beautyHealth']); 16 Route::get('/home-care', [ProductController::class, 'homeCare']); 17 Route::get('/baby-kid', [ProductController::class, 'babyKid']); 18 }); 19 20 // 3. Halaman User (Route Param) 21 Route::get('/user/{id}/name/{name}', [UserController::class, 'profile']); 22 23 // 4. Halaman Penjualan 24 Route::get('/sales', [SalesController::class, 'index']);</pre>
5	Fungsi pada Controller akan memanggil view sesuai halaman yang akan ditampilkan. Sebagai contoh kelengkapan, isi masing-masing file <i>Controller</i> di <code>app/Http/Controllers/</code> dengan fungsi yang me-return View: • HomeController.php<pre>class HomeController extends Controller { 2 references 0 overrides public function index() { return view('home'); } }</pre> • UserController.php<pre>class UserController extends Controller { 1 reference 0 overrides public function profile(\$id, \$name) { return view('user', ['id' => \$id, 'name' => \$name]); } }</pre> • SalesController.php



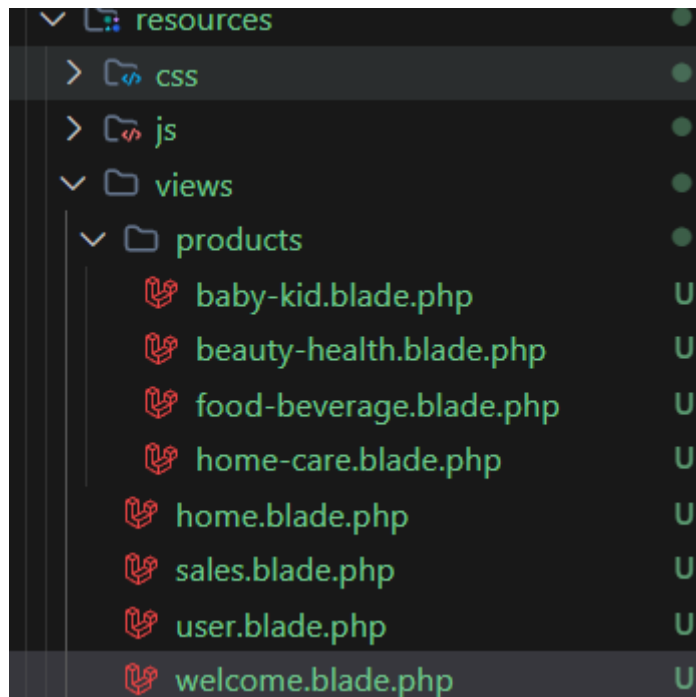
```
POS > app > Http > Controllers > SalesController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6
7  2 references | 0 implementations
8  class SalesController extends Controller
9  {
10     1 reference | 0 overrides
11     public function index()
12     {
13         return view('sales');
14     }
15 }
```

- **ProductController.php**

```
class ProductController extends Controller
{
    1 reference | 0 overrides
    public function foodBeverage()
    {
        return view('products.food-beverage');
    }
    1 reference | 0 overrides
    public function beautyHealth()
    {
        return view('products.beauty-health');
    }
    1 reference | 0 overrides
    public function homeCare()
    {
        return view('products.home-care');
    }
    1 reference | 0 overrides
    public function babyKid()
    {
        return view('products.baby-kid');
    }
}
```

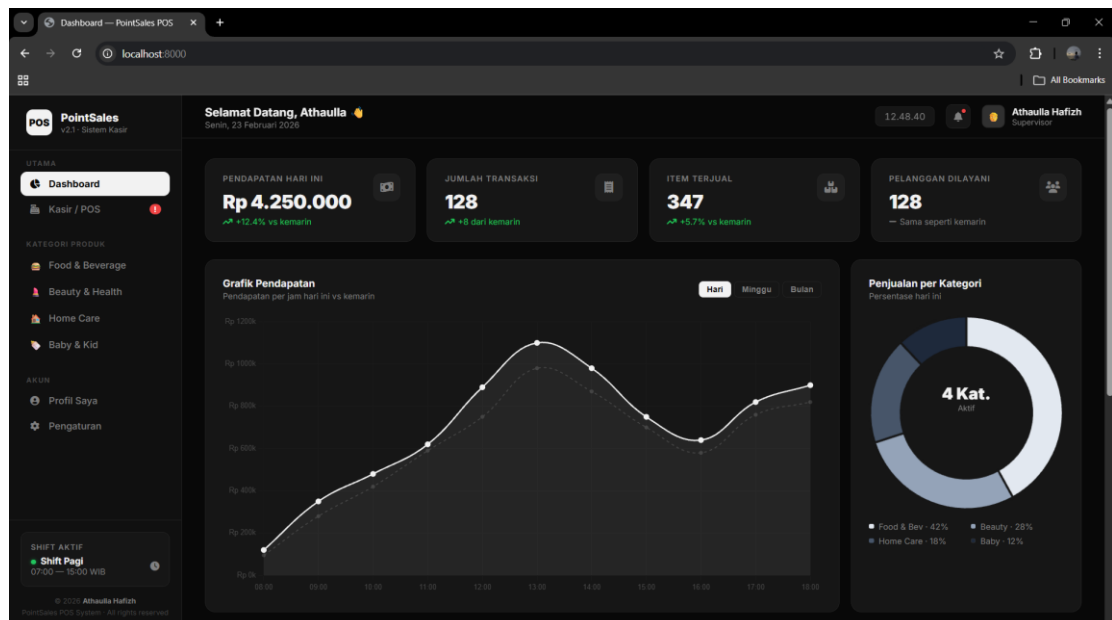


Pembuatan file blade.php



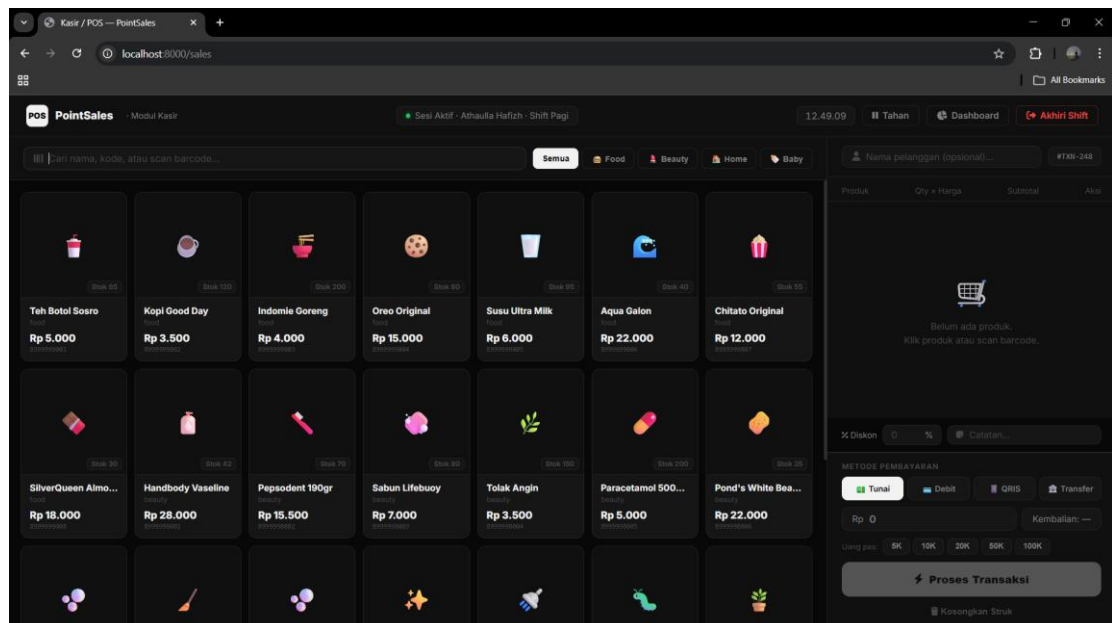
Output

- <http://localhost:8000/> (Akan memunculkan Halaman Home)

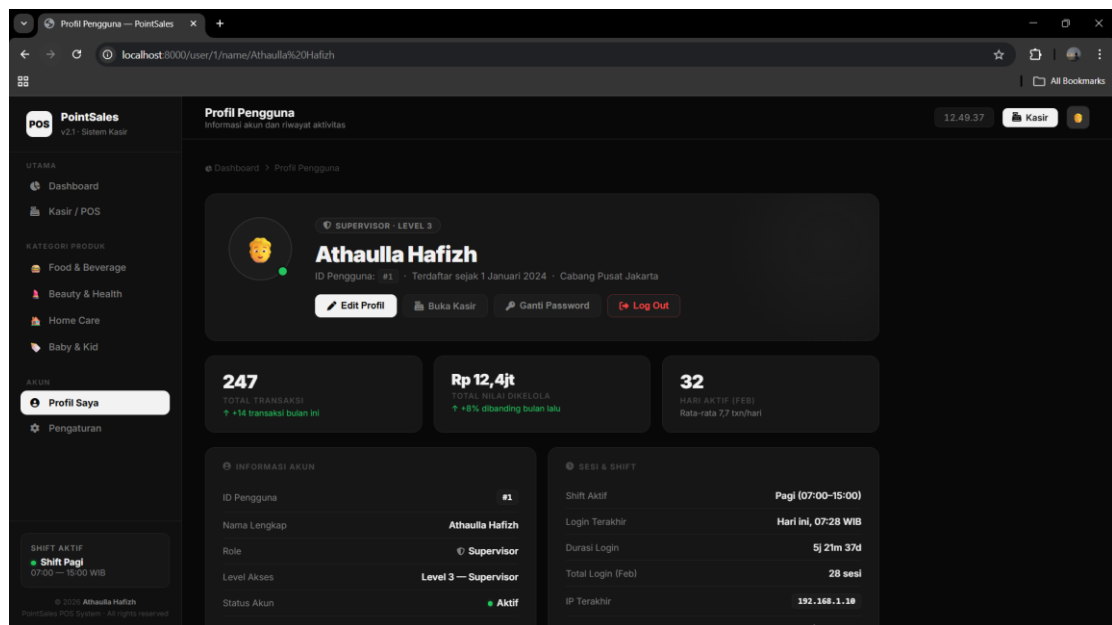




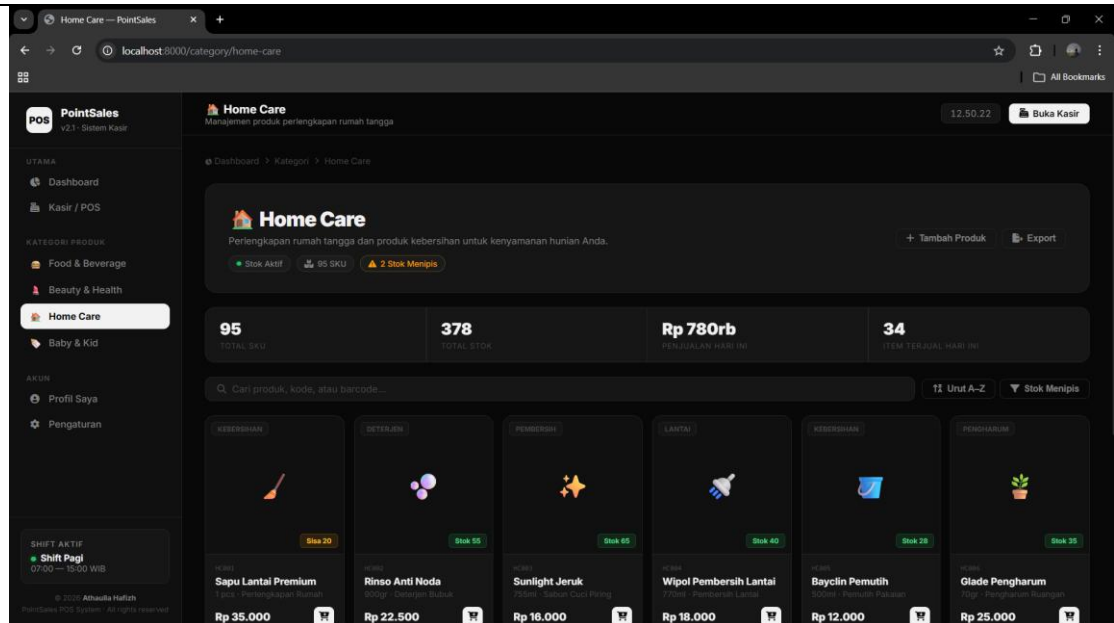
- <http://localhost:8000/sales> (Akan memunculkan Halaman Penjualan)



- <http://localhost:8000/user/> (Akan memunculkan Profil Pengguna beserta ID dan Namanya)



- <http://localhost:8000/category/home-care> (Akan memunculkan kategori produk Home & Care)



6

Simpan setiap perubahan yang dilakukan pada project POS pada Git, sinkronisasi perubahan ke Github.

